

End-to-End Integration and Data Lineage — Operator Manual

Scope: This is the companion to the 20 individual [technology manuals](#). It explains the interfaces *between* them. For exact deployment commands, configuration, and rollback, use the linked component manual. The baseline below is grounded in the [2026-09-17 documentation checkpoint](#) and [cloud continuation checkpoint](#); it is **not** a fresh live health report.

What the system is intended to prove

The project acquires real agricultural sensor observations, transports them over LoRaWAN, stores usable telemetry, and independently preserves/verifies evidence of the gateway-originated data path before eligible evidence is anchored in HRC Hyperledger Fabric. A dashboard picture is not the authoritative measurement or immutable evidence record. Preserve the exact source bytes and the supervised research-recorder output when making research claims.

System boundaries and owners

Boundary	Producer → consumer	Contract / evidence	First diagnosis
Measurement	EMU-01 sensor → RAK5146	RAK4631 firmware payload-v2; accepted profile is OTAA, Class A, plain AS923 , 46-byte application payload	Sensor firmware , RF/Concentrator
RF forwarding	RAK5146/Concentrator → gateway MQTT forwarder	Gateway EUI 0016c001f139a1cb; MQTT region prefix as923	Gateway OS , ChirpStack
Cloud transport	Gateway local persistent Mosquitto → cloud Mosquitto	mTLS bridge over health-gated SIM7600 wwan0 production-primary WAN; Wi-Fi fallback; management Ethernet must not silently become production uplink	LTE , Mosquitto , PKI
LoRaWAN server	Cloud MQTT → ChirpStack	Gateway event, device identity, frame counters, OTAA/session and regional parameters	ChirpStack , Valkey/Sentinel
Application	ChirpStack MQTT integration → Node-RED	Source identity, received payload, decode/validation, measurement mapping	Node-RED
Persistence	Node-RED → PostgreSQL/TimescaleDB	Telemetry writes and atomic Fabric outbox creation; one active Node-RED writer (ULC-03), fenced ULC-02 standby	Node-RED , Database
Evidence	Gateway journal + MQTT witness + application events → evidence services	Raw evidence in SeaweedFS; metadata and verifier-owned results in PostgreSQL	Evidence services , SeaweedFS

Boundary	Producer → consumer	Contract / evidence	First diagnosis
Claim eligibility	Verifier/finalizer → telemetry.fabric_outbox	Immutable finalized_payload IS NOT NULL; v2 additionally requires exactly matching verifier-owned status='verified'	Evidence services , Fabric adapter
Blockchain	Sole enabled adapter on ULC-01 → OpenBao/HRC Fabric	Source-bound contract; exact finalized bytes in transient hrc.exact_payload; authoritative commit, query, digest-match	OpenBao , Fabric adapter
Display/research	Database/service metrics → Grafana/recorder	Grafana for observation; sealed recorder artifacts for counted measurements	Grafana , Research recorder

Follow one physical observation

1. On EMU-01, capture the serial SENSOR_TX marker, firmware identity/profile, observation time, and original application bytes. Do not substitute a synthetic payload without marking the run synthetic.
2. At Gateway-01, distinguish RF reception from MQTT delivery. Confirm the exact regional and device settings *end to end* before modifying radio parameters.
3. At the gateway's local MQTT buffer and LTE route, verify whether the message has reached the bridge. A healthy RF receiver does not prove cloud connectivity; a connected modem does not prove this MQTT flow used LTE.
4. In cloud MQTT and ChirpStack, match gateway/device IDs and the representative event to the application integration. Use frame counters and timestamps appropriately; avoid assuming one event per radio attempt.
5. In Node-RED and PostgreSQL, verify the application record and any derived measurements. Count persisted rows separately from radio transmissions, MQTT deliveries, and blockchain anchors.
6. In the evidence lane, correlate the gateway journal, MQTT witness, raw object, verifier result, and finalization by canonical source identity and observation time. Preserve raw object identity and immutable bytes.
7. In the Fabric outbox, assess eligibility before examining adapter or blockchain health. A pending row with attempts=0 can be correct when finalization/verification has not completed.
8. For eligible claims, verify that only the ULC-01 adapter is write-enabled until the ULC-02 fencing gate passes. Confirm the HRC commit status, QuerySourceBoundAnchor and VerifySourceBoundDigest match; do not equate a submitted proposal with final ledger acceptance.
9. For research results, retain the run manifest, original capture, derived measurements, validation output, and seals/checksums under the protected chapter4-results/ tree using the [research recorder](#) and the current [testing instructions](#).

HRC exact-payload contract

```
Gateway endpoint: 10.104.0.7:7051
TLS server name: peer1.hrc.local
MSP: HrcMSP
Channel: hrc-channel
Chaincode: hrc-evidence
Source namespace: lorawan-gateway-evidence
```

```
CreateSourceBoundAnchor(SourceRecordID, sourceType, producer, producedAt, schemaVersion)
    transient hrc.exact_payload = exact immutable finalized_payload bytes
```

```
QuerySourceBoundAnchor(SourceRecordID)
VerifySourceBoundDigest(SourceRecordID, sha256(finalized_payload))
```

Never reconstruct `finalized_payload` from a JSONB projection; never restore retired transaction names. If submission outcome is uncertain, use the durable prepared transaction and signed commit-status request in reconciling rather than blindly creating another transaction. ULC-02 is a fenced standby until build parity, ownership, and the controlled takeover acceptance are proven.

Failure isolation matrix

Missing result	First boundary to prove	Do not assume
No sensor result	Sensor power/firmware/SERIAL, then AS923 RF	Database or Fabric outage
Gateway hears sensor, no cloud event	Local MQTT, LTE route, TLS bridge, cloud broker	Bad sensor payload
ChirpStack event but no measurement	Integration topic, Node-RED decode/validation, database write	Radio failure
Measurement but no verified evidence	Journal/witness correlation, raw object, verifier/finalizer	Fabric transaction failure
Pending outbox with zero claims	Exact finalized bytes and v2 verifier match	Fabric writer dead
Eligible row not confirmed	Sole writer, DB lease/fencing, OpenBao, Fabric gateway, endorsement, commit, query/digest	A generic network issue
Grafana not showing data	SQL datasource/query/time window, then upstream writer	That no sensor data arrived
Research test not marked ready	Test-specific GO/NO-GO, firmware profile, synchronized clocks, recorder captures	That whole platform is down

Safe service/recovery ordering

For a fresh rebuild, restore network and trust material; etcd; PostgreSQL/Patroni/TimescaleDB and HAProxy/PgBouncer; Valkey/Sentinel and Mosquitto; ChirpStack; OpenBao and evidence/object storage; Node-RED/Grafana; gateway radio, buffered MQTT and LTE; evidence services; and finally Fabric adapter plus real end-to-end verification. Check actual dependencies and roles; this order is not an instruction to restart a working cluster.

For a partial outage, repair the earliest failing layer and verify one representative real operation. Keep buffered gateway data and durable Fabric reconciliation state. Never purge research evidence, delete journal lineage, clear the outbox, enable a second writer, or invoke failover to make a dashboard temporarily green.

Minimum acceptance evidence

- Actual firmware/profile, sensor, Gateway EUI, AS923 settings and original payload identified.
- Sensor RF → local MQTT → cloud MQTT → ChirpStack → application → database traceable for one real observation.
- Verifier-owned matching evidence and raw object available for v2 claims.
- If anchoring is tested, sole adapter writer and authoritative Fabric commit/query/digest result confirmed.

- Timestamps, collection method, units, test mode and checksums in the retained research artifact.
- No immediately relevant critical error; explicitly state any layer that could not be tested.

See [PDF publication and review](#) before releasing this guide or the 20 technology manuals as PDFs.

Supporting runbook: 00-current-server-continuation-checkpoint.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Current Server / Gateway / Sensor State

Authoritative current-state summary through 2026-09-17. This file is intentionally concise. It records the latest verified operating boundary only; commissioning chronology, superseded values, failed attempts, and dated debugging detail belong in [00-build-execution-log.md](#) or the relevant component manual.

Operator entry points

- Research execution and evidence capture: [../.../test/00-README.md](#), [../.../test/automation/research-recorder/README.md](#), then the selected manual under [../.../test/execution/](#).
- Cloud deployment/operations: this file, then [00-README.md](#) for the component map.
- Detailed historical commissioning evidence: [00-build-execution-log.md](#). Do not use old intermediate status statements there as current truth.
- Historical plans, raw thesis extracts, and completed bring-up handoffs are under [../.../docs/archive/](#) and are not operator authorities.

Fresh pre-test readiness scan - 2026-09-16

A new non-destructive end-to-end readiness scan was executed on 2026-09-16 before counted research work. The repository readiness gate `test/automation/research-manual/ensure_ready.py` completed with `TECHNICAL_GATE=PASS`, `TOOL_COMPILE=PASS`, `MQTT_TOOL_SELFTEST=PASS`, `COUNTED_FIRMWARE=PASS`, `RECORDER_STATE=CLEAN`, `GATEWAY_LTE=PASS`, and `LIVE_PRE=PASS` at 2026-09-16T07:05:21Z.

Fresh verified facts from this scan:

- EMU-01 is present on COM11; SEC is present on COM16.
- The archived 15-second counted EMU-01 profile is reproducible and hash-consistent with the current tracked source. The boot capture recorded `EMU01_SENSOR_INIT=PASS`.
- Gateway-01 LTE is up on `wan0`; the metric-10 default route and the route to the production MQTT endpoint use LTE; both MQTT TLS bridge sockets are established; the SIM7600 is registered on LTE.
- ULC-01/02/03 and Gateway-01 are reachable through the restricted research-recorder path.
- PostgreSQL still presents exactly one leader in the recorder preflight.
- Gateway-evidence collector/verifier readiness checks are HTTP 200 on their commissioned nodes.
- The latest accepted gateway checkpoint for `0016c001f139a1cb` was visible during the live preflight, and no `evidence_gap / integrity_failure` condition caused the gate to fail.
- Cross-host timing passed the recorder's strict readiness limits: cloud spread remained below 250 ms and gateway/cloud skew remained below 1.5 s.

This is the current **technical platform readiness** boundary. It does not replace the formal sensor GO/NO-GO procedure in `test/preparation/sensor/preflight/04-go-no-go-transition.md`. Before a counted full-stack run,

preserve fresh evidence for the final sensor build: OTAA JoinRequest/JoinAccept, at least ten consecutive post-join uplinks with zero selected codec errors, source-to-decoder comparison, application/TimescaleDB correlation, and the required Fabric/outbox condition. The current live PRE intentionally does not fabricate those research observations.

The test-command compatibility matrix is also intentionally fail-closed: PRE and P1 are fully READY; several later experiments remain CAPTURE_READY or HARNESS_REQUIRED, and S2 remains METHODOLOGY_REQUIRED. Those labels are experiment-orchestration/methodology gates, not failures of the commissioned LoRaWAN infrastructure.

Research/Fabric eligibility diagnostic - 2026-09-17

The current adapter implementation and research diagnosis add one important operator rule: a Fabric outbox row can remain pending with attempts=0 even while the ULC-01 writer is healthy. The normal claim query requires immutable finalized_payload IS NOT NULL; telemetry-attestation-v2 additionally requires a matching verifier-owned gateway_evidence.event_verification.status='verified' row. Diagnose outbox finalization/eligibility before treating an unclaimed row as a Fabric, OpenBao, or worker outage.

The commissioned HRC Task 37 API remains source-bound: CreateSourceBoundAnchor(SourceRecordID, sourceType, producer, producedAt, schemaVersion) with exact immutable finalized_payload bytes sent only as transient hrc.exact_payload, followed by QuerySourceBoundAnchor(SourceRecordID) and VerifySourceBoundDigest(SourceRecordID, observedDigest). Prepared transaction bytes and a signed commit-status request are persisted before orderer submission; uncertain outcomes enter governed reconciliation rather than blind resubmission. A stale HRC peer ledger can make client CommitStatus waiting appear hung even when endorsement and orderer submission succeeded, so troubleshoot those layers separately.

Audit note: a fresh direct SSH probe from the documentation workspace on 2026-09-17 was denied by public-key authentication. This audit therefore makes no new host-runtime claim from that failed probe; it reconciles current facts from commissioned checkpoints, research preflight evidence, and current implementation/source contracts.

Current operational state

Layer	Current verified state
etcd	3-node quorum commissioned
PostgreSQL / Patroni / TimescaleDB	HA cluster commissioned; application and evidence database paths proven
HAProxy / PgBouncer	Commissioned verify-full database routing and role authentication
Mosquitto	Preferred/backup HA normal path commissioned; do not restore round-robin live-session routing
Valkey / Sentinel	HA path commissioned
ChirpStack	Two-node cloud cluster commissioned on plain AS923 / MQTT prefix as923
OpenBao	3-node KMS normal path and audit commissioned
Node-RED	u1c-03 A active; u1c-02 B fenced/standby; atomic telemetry + Fabric outbox path proven
Grafana	Server deployment and real telemetry/evidence read path commissioned

Layer	Current verified state
Gateway evidence	Gateway journal/uploader, replicated ingest/collectors/verifiers, trusted decoder, and object-storage path commissioned
Gateway-01	RAK5146, AS923, SIM7600 LTE primary backhaul, Wi-Fi fallback, management Ethernet, MQTT and evidence paths physically proven
EMU-01	OTAA, payload-v2, production scheduler, application telemetry, and evidence lineage physically proven
Fabric	ULC-01 production writes enabled and verified ; ULC-02 remains write-disabled until HA ownership/fencing acceptance
Public ingress	ChirpStack/Evidence/MQTT normal path commissioned on Reserved IPv4; provider-side reassignment/failover acceptance remains a separate gate
Research recorder	Core supervised capture path operational; fixed aggregate evidence-status wrapper extension remains a separately staged compatibility improvement where not yet deployed

The current system is ready for the research procedures whose own GO/NO-GO gates pass. Do not recommission healthy infrastructure merely because an older dated section, plan, or build log describes a previously pending state.

Fresh-start gateway evidence lineage baseline - 2026-09-11

A new deliberate evidence/test reset was completed at approximately 2026-09-11T02:44Z after a live acceptance run exposed that Gateway-01 still retained its commissioned historical journal while the server metadata had previously been cleared. The operator explicitly requested that the Gateway-01 segment lineage be deleted and restarted from a true fresh origin.

Verified reset boundary:

- Gateway-01 evidence writer, uploader, and Mosquitto bridge were stopped before mutation.
- Gateway-01 had 226 closed journal segment files plus 1 open segment, with durable state `next_segment_id=227`, `next_sequence=3224`. All closed/open journal files, `journal-state.json`, and uploader receipt state were removed. Evidence TLS material, UCI configuration, firmware, and identities were preserved.
- The seven active research/runtime PostgreSQL tables were truncated with identities reset: `telemetry.uplinks`, `telemetry.measurements`, `telemetry.fabric_outbox`, `gateway_evidence.mqtt_gateway_events`, `gateway_evidence.event_verification`, `gateway_evidence.segments`, and `gateway_evidence.checkpoints`. The pre-reset counts were 9, 117, 9, 271, 9, 4, and 4 respectively; the transaction completed with all seven at 0.
- Immutable SeaweedFS/raw object-store history was intentionally preserved. It has no active segment/checkpoint metadata references after this reset and must not be treated as current lineage.
- Gateway services were restarted in writer -> uploader -> Mosquitto order. The new durable journal state is exactly `next_segment_id=1`, `next_sequence=1`, `previous_record_hash=GENESIS`, `previous_segment_hash=GENESIS`, with 0 closed and 0 open segment files immediately after initialization.
- The uploader recreated an empty canonical receipt state: `{"checkpoint_receipts": {}, "segment_receipts": {}}`.
- After the MQTT bridge reconnected, fresh gateway stats began arriving normally. At the first post-reset consistency check: `telemetry.uplinks=0`, `telemetry.measurements=0`, `telemetry.fabric_outbox=0`,

gateway_evidence.event_verification=0, gateway_evidence.segments=0, and gateway_evidence.checkpoints=0; gateway_evidence.mqtt_gateway_events=5 represented only new post-reset gateway traffic.

- Because the DITO uplink remained unavailable during this acceptance session, the already-established temporary opaque workstation relays for MQTT TLS and evidence HTTPS/mTLS remained in service. TLS and client-certificate authentication remain end-to-end; these relays are transport-only and are not a permanent architecture change.

This 2026-09-11 boundary supersedes the prior Gateway journal lineage for new research testing. New evidence must begin from segment 1 / sequence 1 / GENESIS. Do not replay the deleted Gateway receipt/journal state into the active runtime.

Fresh-start testing baseline - 2026-09-10

A definitive active-runtime reset began at 2026-09-10T06:11:31Z and was re-verified through 2026-09-10T06:21:33Z. Both ChirpStack instances, both server Mosquitto brokers, Node-RED, the evidence ingest/collector/verifier workers, and both Fabric adapter instances were quiesced before state was cleared.

The reset deliberately established this empty testing epoch:

- telemetry.uplinks, telemetry.measurements, telemetry.fabric_outbox, gateway_evidence.mqtt_gateway_events, gateway_evidence.event_verification, gateway_evidence.segments, and gateway_evidence.checkpoints were truncated with identities reset; final counts remained exactly 0 on all three PostgreSQL members.
- Valkey DB 0 on the ULC-02 primary was reduced from 37 legacy ChirpStack metrics/stream keys to 0, synchronously, and the empty state was saved and replicated. Those keys were prior metrics/stream runtime history rather than device registration/configuration.
- The server Mosquitto persistence databases on ULC-01 and ULC-02 were backed up and removed while the brokers were stopped. After the complete application stack had restarted, neither broker had recreated mosquitto.db, proving there was no persisted server-broker queue/session residue at the final check.
- Node-RED was restarted, clearing transient in-memory state. Its tracked/runtime flow configuration under /srv/node-red/data was preserved.
- ChirpStack registration state was intentionally preserved: tenant=1, application=1, device=1, gateway=1; device_queue_item=0 and multicast_group_queue_item=0 remained empty.
- Immutable SeaweedFS raw evidence objects, OpenBao audit/security history, configuration, identities/certificates, schemas, deployment material, and research/recovery archives were preserved. They are audit/recovery history, have no active telemetry/evidence database references after the reset, and cannot replay themselves into the live pipeline.

Fresh-start recovery copies:

```
ULC-01 /var/backups/lorawan-clean-reset/freshstart-20260910T061131Z/mosquitto-ulc01.db
ULC-02 /var/backups/lorawan-clean-reset/freshstart-20260910T061131Z/mosquitto-ulc02.db
ULC-02 /var/backups/lorawan-clean-reset/freshstart-20260910T061131Z/valkey-ulc02-preflush.rdb
```

Final health proof after the reset: ULC-01 remained Patroni primary with ULC-02/03 streaming; the primary etcd quorum and the separate SeaweedFS metadata-etcd quorum answered healthy commit probes; both ChirpStack frontends returned HTTP 200; evidence collectors and verifiers reported ready; Node-RED was Docker-healthy and HTTP 200; Grafana /api/health reported database ok; OpenBao was initialized/unsealed with ULC-02 active; SeaweedFS master endpoints returned HTTP 200; ULC-01 reached 10.104.0.7:7051 and logged fabric_adapter_ready with its canonical self-test passing; ULC-02 Fabric remained correctly fenced in standby. Every server had zero failed systemd units and zero exited Docker containers. MQTT/TLS reconnect warnings

observed only while the brokers were deliberately stopped disappeared after stabilization, with zero post-stabilization application errors.

This 2026-09-10 boundary is the authoritative empty testing epoch. Any telemetry/evidence rows or ChirpStack runtime metric/stream keys created after it are fresh post-reset data.

Prior clean server runtime baseline - 2026-09-09

A deliberate server-side clean reset was completed after the September 9 research/presentation work. Writers were quiesced first, the full `lorawan_telemetry` database was archived from the verified Patroni leader, and only active telemetry/evidence runtime rows were truncated. ChirpStack tenant/application/device/gateway registrations, database schemas, credentials, certificates, deployment files, persistent volumes, tagged rollback images, and `chapter4-results/` research evidence were preserved.

Recovery archive on ULC-01:

```
/var/backups/lorawan-clean-reset/lorawan_telemetry-preclean-20260909T095657Z.dump
SHA-256 382723d51f126f0a555368a12bc15840664229e2e43004fba4e0c5a27c767580
/var/backups/lorawan-clean-reset/lorawan_telemetry-preclean-20260909T095657Z.manifest.txt
```

The post-clean replicated baseline is exactly zero rows in `telemetry.uplinks`, `telemetry.measurements`, `telemetry.fabric_outbox`, `gateway_evidence.mqtt_gateway_events`, `gateway_evidence.event_verification`, `gateway_evidence.segments`, and `gateway_evidence.checkpoints` on ULC-01/02/03. ULC-01 is the Patroni primary; ULC-02 and ULC-03 are streaming replicas at the same WAL location. Evidence collectors/verifiers are ready, the ULC-01 Fabric adapter is enabled/ready, the ULC-02 adapter is standby, Node-RED is healthy, both ChirpStack frontends return HTTP 200, Grafana health reports its database ok, and OpenBao is initialized/unsealed on all three nodes with ULC-02 active at this check. No exited Docker containers or failed systemd units remained. Old application Docker logs, closed `/tmp` deployment/test artifacts, the stale `grafana-pre-pgbouncer-fix-20260909` container, and disposable Docker build cache were removed; active configuration and recovery assets were not pruned.

Fabric production boundary

ULC-01 passed the HRC Task 37 external qualification and is the only production-enabled Fabric writer at this boundary.

Verified integration anchors:

```
FABRIC_GATEWAY_ENDPOINT=10.104.0.7:7051
FABRIC_TLS_SERVER_NAME=peer1.hrc.local
FABRIC_MSP_ID=HrcMSP
FABRIC_CHANNEL=hrc-channel
FABRIC_CHAINCODE=hrc-evidence
AuthenticatedSourceSystemID=lorawan-gateway-evidence
ULC-01 FABRIC_ADAPTER_ENABLED=true
ULC-02 FABRIC_ADAPTER_ENABLED=false
```

Qualification row 1960 is confirmed on Fabric transaction

`dd11910c1e7c28edd11334a03cce10d80d574692733458c9059d2775be5131b8`; source-bound query and digest verification matched. On 2026-09-09 the ULC-01 production Adapter passed its activation preflight, had zero restarts, and had produced 325 post-activation confirmed writes (326 total including the qualification row). The verified immutable image ID is `sha256:d12e73ae24f7823730b6632fe8209e844c0d6125bd6a9c13e9d75cc330664f74`; service-binary SHA-256 is `a53990277d9032e60a9f0bfb52619a21e8c999a3573477dbfb3efe6ca6fb9591`.

Before the 2026-09-09 clean reset, no eligible verified exact-payload rows were waiting; the historical pending/dead-letter backlog and previously retained verifier-pending rows were runtime history, not replay candidates. That state is no longer present in the active database: it is preserved only in the pre-clean recovery dump and manifest listed above. Do not restore or replay that dump into production except for an explicit recovery/investigation, and do not rerun qualification candidate 1960.

ULC-02 activation is a distinct HA boundary. Keep it `FABRIC_ADAPTER_ENABLED=false` until the documented ownership/fencing and failover conditions are deliberately accepted. Do not reuse HRC administrator material, the Task-39 writer, Caliper aliases, or temporary qualification credentials.

Gateway and radio boundary

Gateway-01 is physically accepted on the commissioned AS923 image and configuration. The accepted image remains:

```
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
SHA-256 bafe8b97baf9353df2654b1c8b71fa53d2ff764cd264d0ed6c924dd25a5ec67d
Gateway EUI 0016c001f139a1cb
```

The live gateway has proven RAK5146 detection, intended EUI, AS923 radio operation, reboot persistence, protected identity restore, real EMU-01 traffic, evidence lineage, and SIM7600 LTE cloud transport. LTE is the health-gated production primary route, Wi-Fi is automatic fallback, and Ethernet remains management-only. Rebuild or reflash only for an actual regression or intentional image change.

EMU-01 uses the tracked source under `../../../../firmware/EMU01_Agriculture_Node/`. The accepted application contract remains payload-v2, 46 bytes, OTAA, Class A, plain AS923, ADR, unconfirmed normal uplinks, and deterministic `SENSOR_TX` serial output. Production scheduling uses local sampling plus approximately five-minute staggered/jittered uplinks; temporary counted-test firmware is a research profile, not the production baseline.

Evidence and application boundary

The accepted normal path is:

```
EMU-01 -> LoRaWAN RF -> Gateway-01 -> LTE/mTLS MQTT -> ChirpStack
        -> Node-RED -> TimescaleDB -> Grafana
        -> telemetry.fabric_outbox -> Fabric Adapter -> OpenBao seal/verify -> HRC Fabric

Gateway journal + MQTT witness + application telemetry
        -> replicated evidence ingest/collectors/verifiers -> trusted decoder/object storage
```

Gateway evidence is part of the security proof, not optional logging. Keep the journal/uploader, mTLS ingest boundary, collectors, verifier, trusted decoder, and retained research evidence intact. `chapter4-results/` contains research artifacts and must never be treated as disposable documentation clutter.

Remaining boundaries

These are not reasons to rebuild already-passing layers:

1. **Fabric HA:** ULC-02 writer activation/failover remains gated by its ownership/fencing acceptance.
2. **Reserved IPv4 HA:** normal public ingress is proven; provider-side reassignment authority and controlled failover acceptance remain separate work.
3. **Research outbox finalization/eligibility:** current v2 research rows must receive immutable `finalized_payload` and satisfy verifier-owned verified state before the Fabric worker can claim them; fix

this upstream boundary rather than masking it with manual Fabric retries.

4. **Backup/recovery acceptance:** execute only the remaining acceptance items identified by the backup/recovery manual; do not repeat already-recorded normal-path checks without cause.
5. **Research execution:** follow the current test/ GO/NO-GO gates and table-by-table procedures; preserve sealed run artifacts.
6. **Recorder wrapper version skew:** where the restricted server wrapper does not yet expose aggregate evidence-status, the recorder intentionally records UNAVAILABLE_WRAPPER_UPGRADE_REQUIRED rather than fabricating counts. This does not invalidate independent readiness checks that already pass.

Do not repeat without a state-changing reason

- Core HA commissioning already proven.
- Gateway flashing, RAK5146 detection, AS923 commissioning, LTE primary-path proof, and EMU-01 normal RF acceptance.
- Task 37 Fabric qualification row 1960.
- Broad replay of historical Fabric outbox rows.
- Disruptive failover tests after unrelated documentation or presentation changes.
- Any use of archived plans/handoffs as if they were current runbooks.

Authoritative component references

- Cloud component map: [00-README.md](#)
- Historical commissioning evidence: [00-build-execution-log.md](#)
- Fabric/OpenBao operations: [20-openbao-and-fabric-adapter.md](#)
- Backup/recovery: [13-backup-restore-and-disaster-recovery.md](#)
- Troubleshooting: [17-troubleshooting.md](#)
- Gateway setup: [.././gateway/setup/](#)
- Research testing: [.././../test/00-README.md](#)
- Research evidence layout: [.././../chapter4-results/README.md](#)
- Historical/provenance material: [.././../docs/README.md](#)

Supporting runbook: README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Automated Research Recorder

This is the preferred evidence-capture path for counted dissertation tests on the **commissioned** Gateway-01 + LTE + three-node cloud system.

Evidence authority

The testing evidence is authoritative for observed results. The current methodology authority for what must be tested and measured is [.././ZACARIAS-CHAPTER3-TEST-METRICS-SOURCE-OF-TRUTH.md](#), derived from the new chapters/Zacarias_Chapter3.pdf. Use it for research questions, experimental conditions, counts, durations, measurements, analysis rules, and stated acceptance/security criteria. Do not treat architecture, topology, technology placement, host roles, status statements, or expected outcomes in that PDF—or older dissertation drafts—as current infrastructure truth.

Use this order when sources disagree:

1. sealed evidence captured for the specific formal trial under chapter4-results/ is the authority for that trial and its research classification;
2. fresh read-only live observations are the authority for current operational state at the time they were observed;
3. current deployment/configuration material is the reproducibility reference;
4. chapter PDFs, old DOCX files and historical prose are requirements/provenance context only.

A healthy live service does **not** prove that a research trial passed. The recorder deliberately captures evidence first and leaves classification to the evidence from that trial. The implementation must follow the live commissioned system rather than recreating obsolete architecture described in older drafts.

What is automated

One recorder run automatically retains:

- EMU-01 USB serial with host UTC timestamps and automatic COM-port re-enumeration recovery;
- optional SEC USB serial capture;
- Gateway-01 live logread stream;
- Gateway-01 CPU, memory, load and network counters;
- Gateway-01 LTE/QMI, route and production MQTT-socket snapshots;
- host CPU, memory, load and network counters on ulc-01, ulc-02, ulc-03;
- per-container CPU/memory series on all three cloud nodes;
- exact-window ChirpStack, Node-RED, Mosquitto and kernel logs;

- authoritative telemetry.uplinks, telemetry.measurements, and telemetry.fabric_outbox CSV exports from the current Patroni leader;
- RSSI, SNR, gateway frequency, gateway uplink ID, FCnt and test_sequence statistics;
- byte-exact EMU-01 source-to-database **delivery correlation** whose source timestamps can be clock-aligned into the authoritative DB export window; this is not the formal Chapter 3 PDR;
- formal Chapter 3 PDR is reported only when the numerator can be derived as unique legitimate uplinks accepted by ChirpStack over the same eligible source-attempt window;
- application-event-to-database timing (received_at - time) plus independently calibrated **sensor-transmission-to-database-storage latency** from matched SENSOR_TX source timestamps when recorder-v9 start/end clock calibration passes;
- a read-only aggregate verifier/checkpoint snapshot at run start and run end (metadata/evidence-status-start.txt and metadata/evidence-status-end.txt);
- SHA-256 for every completed evidence file.

Raw files are made read-only after finalization. Calculated files live under derived/; never edit raw evidence to improve a result.

Live testing monitor

research_cockpit.py provides the read-only testing monitor at <http://127.0.0.1:8765/>. It is an observation aid for running experiments, not a second result engine.

The cockpit combines the existing live sensor/resource view with testing_monitor.py. Each cloud node uses one locked-down SSH monitor-stream session so monitoring does not create a burst of competing SSH probes during a trial. Resource records remain approximately one-second observations; slower service/database/evidence observations are multiplexed through the same session. The server wrapper discovers the live Docker/service set instead of relying on chapter-era service lists.

The monitor shows:

- current CPU utilization as percent of 100%, RAM used/total, and network receive/send rates in Mbps with explicit capacity context where a defensible capacity is known;
- received sensor packets by sender, DevEUI, gateway, AS923 radio frequency, RSSI in dBm, SNR in dB, FCnt/test sequence and decoded sensor values;
- packet-level RF-to-application correlation from Gateway-01 reception through gateway MQTT evidence, ChirpStack/application acceptance, TimescaleDB, evidence verification, Fabric outbox and Fabric transaction fields when present;
- duplicate/replay indicators that distinguish an exact PHYPayload received again under a new gateway uplink ID from ordinary MQTT redelivery, plus FCnt/test-sequence repeats, gaps and regressions;
- discovered containers and selected system services with node, role/state and live detail;
- current Patroni leader/replica observations;
- evidence collector/verifier and Fabric adapter readiness, including the deliberately fenced Fabric standby when observed;
- leader-only database counters for LoRaWAN uplinks, decoded measurements, gateway MQTT evidence, evidence segments, checkpoints, verifications and Fabric outbox rows;
- recent warning/error observations and the formal test inventory from chapter4-results/.

Operational observation history is stored locally in %LOCALAPPDATA%\LoRaWAN\testing-monitor.sqlite3 using SQLite/WAL. Packet observations are stored separately in %LOCALAPPDATA%\LoRaWAN\packet-monitor.sqlite3. Service snapshots are persisted at a bounded cadence (currently 60 seconds), packet correlation is refreshed every five seconds, the UI exposes recent history, and observations are retained for seven days. These local

histories are useful for diagnosing service and packet behavior; they are **not** substituted for the sealed raw/metadata/derived files of a formal trial.

The packet monitor deliberately uses three different duplicate concepts. An exact `phy_payload_sha256` observed with a **new gateway uplink ID** is a strong RF replay/duplicate candidate. Re-observation of the same gateway uplink ID is transport/capture redelivery and is not counted as another RF packet. Duplicate application identity, `FCnt` or `test_sequence` is a downstream/session anomaly and is reported separately. This distinction prevents QoS 1 MQTT behavior from being misreported as a LoRaWAN replay.

Clock and timing gate

Counted runs fail closed unless the source-clock relationship to authoritative cloud UTC is **measured and stable**. `serial_capture.py` timestamps each EMU-01 `SENSOR_TX` line with the workstation clock, while the authoritative run window and database evidence use cloud/server UTC. Recorder v9 therefore does not assume that the workstation wall clock is correct: it independently calibrates the workstation against `ulc-01` immediately before the formal window, repeats the calibration after collectors stop, and invalidates finalization if the start/end mapping drifts by more than **0.100 s**. The calibrated mapping is retained in `metadata/run-meta.json` and is used to derive the Chapter 3 sensor-transmission-to-database-storage latency.

The recorder preflight still takes five midpoint-corrected `utc-now` samples from each of `ulc-01`, `ulc-02`, and `ulc-03`. The three cloud clocks must agree within **0.250 s**, Gateway-01 must remain within **±1.500 s** of the cloud median, and the workstation/cloud offset must be measurable within the bounded calibration range (**±300 s**). Absolute workstation offset is recorded rather than silently treated as zero. A large offset is acceptable only when start/end calibration remains stable; cloud disagreement, gateway disagreement, missing calibration, excessive drift, or negative calibrated sensor-to-database latency invalidates timing evidence.

Historical staging diagnosis found that the domain workstation could inherit a large absolute offset from its PDC even while the cloud and gateway clocks remained mutually coherent. Recorder v9 records that offset explicitly and proves its stability at both run boundaries, so a large but stable workstation wall-clock offset no longer blocks a run or contaminates source timing. The Active Directory time source should still be repaired through the normal PDC hierarchy for operational correctness; do not point a domain member directly at public NTP merely to satisfy research tooling.

Gateway-01 remains independently gated against the cloud median because gateway radio/evidence timestamps are part of the cross-system observation boundary. Its limit remains **±1.500 s**. If gateway/cloud agreement or cloud-node spread exceeds its limit, the recorder still fails closed. The restricted research-recorder key remains read-only and must not be weakened to repair time service configuration.

The cockpit's **Overall staging readiness** panel combines this clock gate with live cloud streams, Gateway management visibility, Patroni role topology, evidence readiness, leader database counters, telemetry-read availability, and the absence of an already-active formal recorder. This panel is an operational preflight aid only; the recorder CLI remains the final start gate and additionally checks required local USB/serial devices.

The server side remains read-only. `DBMETRIC` records are emitted only by the node that is observed as the PostgreSQL leader, avoiding duplicate HA counts. If a gateway management observation becomes unavailable, the cockpit shows **Management observation unavailable** and **Not observed** for unavailable gateway metrics; this means the management path could not be observed and does not prove the gateway or LoRaWAN radio path is offline. Sensor telemetry has separate semantics: a successful database read with zero rows is `idle` / **No fresh packets**; a failed live query is **Telemetry observation unavailable**; a transient read failure with a previous valid snapshot retains that snapshot as `stale`; only a read failure with no usable snapshot is an `error`. The clean fresh-test baseline is therefore expected to show **No fresh packets** until real new AS923 uplinks arrive. Telemetry queries use bounded five-second reads and a 30-second cache horizon, and their SSH subprocesses explicitly use a closed stdin (`DEVNULL`) so a detached/managed cockpit worker cannot block waiting on inherited console input. These rules prevent a slow polling cycle or unavailable management path from being mislabeled as device failure.

Recorder v4 adds a **foreground supervisor**. `run` is now the preferred Chapter 4 entry point: it starts the same serial/remote collectors as `start`, performs a bounded collector/evidence health gate every 15 seconds by default, records those observations in `raw/control-plane-health.csv`, and invokes the normal `export/summarize/hash` finalizer automatically on duration completion or `Ctrl+C`. This prevents an unnoticed workstation-session loss from leaving stale active state and silently invalidating later trials. Each supervisor row records the server-authoritative UTC timestamp, the workstation midpoint timestamp around that clock probe, the measured server-minus-workstation offset, and probe round-trip time. This makes workstation clock skew explicit instead of silently mixing clock domains. Experiment timing still uses server-authoritative timestamps and source/database evidence as described below.

Recorder v5 hardens that supervisor for real multi-minute counted work. A dead collector process remains an immediate terminal failure because evidence is actually being lost. Short-lived remote-readiness, USB/re-enumeration, database-role, or clock-probe failures are instead retained as `WARN` observations and must persist for three consecutive health gates before the supervisor aborts; a later clean gate resets the transient-failure count. Per-node SSH/readiness timeouts are converted into explicit health details rather than uncaught exceptions. If the server-authoritative clock probe is temporarily unavailable, the CSV leaves the authoritative timestamp and skew fields blank, records the condition as a warning, and never substitutes workstation time as if it were server time. `status`, `stop`, and `recover` also tolerate a failed final health probe so preservation/finalization can still complete. This policy was added after the 2026-09-04 uncounted counted-profile reload capture showed that a single transient readiness timeout could terminate the foreground supervisor while all child collectors and the actual LoRaWAN path remained healthy.

Workstation dependency

The recorder is Python-native because this workstation is governed by a domain PowerShell `MachinePolicy=Restricted`. Do not weaken that policy.

Python dependency is pinned in `requirements.txt`:

```
pyserial==3.5
```

Install/reproduce with:

```
python -m pip install --user -r .\lorawan-network-server-gateway\test\automation\research-recorder\requirements.txt
```

The older `.ps1` prototypes in this directory are not the operator entry point on this workstation.

Locked-down SSH boundary

The recorder uses a dedicated key outside the repository:

```
%USERPROFILE%\ssh\id_ed25519_research_recorder
```

The private key is never stored in the project. Its public key is installed on the three cloud nodes and Gateway-01 with a forced-command wrapper. The key can invoke only the `read/capture` verbs implemented under `remote/`; an arbitrary command such as `uname -a` is rejected.

Never replace this with an embedded SSH password or copy an administrator private key into the repository.

Forced-command monitor boundary - current state 2026-09-18

Fresh restricted-key version checks on ulc-01, ulc-02, and ulc-03 all return research-recorder-server-v7, matching the current repository wrapper. Do not reuse the historical v5 wrapper hash as a v7 integrity claim; verify the deployed file hash separately whenever byte-for-byte wrapper attestation is required. Gateway-01 remains on its separately versioned restricted gateway wrapper. The dedicated recorder key remains forced to reviewed verbs; arbitrary shell commands are not permitted.

Server wrapper v7 retains the bounded recorder/readiness verbs, `monitor-stream <seconds>`, and the fixed read-only `db-export packetflow <startUTC> <endUTC> <gatewayEUI>` projection used by the live packet correlator. The monitor stream multiplexes one-second host-resource records with slower service, readiness, database-role, database-counter and bounded recent-error observations. Optional service/error side probes are deliberately non-fatal, so an empty error scan or one failed auxiliary observation cannot terminate the resource stream and silently leave the cockpit stale. The packetflow projection joins already-authoritative gateway MQTT evidence to accepted telemetry, evidence-verification status, Fabric outbox state and Fabric transaction fields. It accepts no arbitrary SQL, table name, column name or filter expression from the caller.

Gateway wrapper v2 retains snapshot, resource and logstream and adds bounded `radio-recent <count>` and `journal-recent <count>` reads. `radio-recent` exposes only recent RAK5146 frame-reception log lines. `journal-recent` exposes retained immutable gateway journal records needed to correlate exact PHYPayload bytes for authorized replay testing. Neither command changes radio, network or journal state.

The version verbs and the fixed packetflow/radio reads were verified through the same restricted recorder identity after deployment. Keep the repository wrappers and deployed copies converged; do not bypass or weaken the forced-command boundary to troubleshoot monitoring.

Read-only evidence-status boundary

The bounded evidence-status capability remains part of the current wrapper. It executes fixed SELECT-only queries against the commissioned evidence status views and returns aggregate verification/checkpoint state only. It exposes no payload bytes, evidence hashes, database credentials, arbitrary SQL, or arbitrary shell access.

The older `UNAVAILABLE_WRAPPER_UPGRADE_REQUIRED` compatibility behavior is historical only. Current cloud nodes run wrapper v7, so new formal runs are expected to capture the fixed evidence-status output when the underlying evidence read path is healthy. If a current run receives `command not allowed` or an `unavailable-wrapper` marker from a cloud node, treat that as a deployment/version regression and investigate it rather than accepting it as the present baseline.

Journal-read prerequisite

opsadmin must be a member of `systemd-journal` on ulc-01, ulc-02, and ulc-03. This is required so the forced-command recorder identity can retain Mosquitto and kernel-journal evidence without using `sudo` or an unrestricted administrator shell. The membership was applied and verified on all three cloud nodes on 2026-09-03.

Accepted recorder smoke - 2026-09-03

The first uncounted smoke, `recorder-smoke-20260903`, proved the core `start/capture/stop/export/summarize/hash` path but exposed the missing journal-read permission above; preserve that run as diagnostic evidence. After correcting journal access, `recorder-smoke-journal-20260903` completed cleanly with all eight live collectors healthy, recorder stop exit 0, status `RECORDED_UNCLASSIFIED`, no stop-error file, 65 SHA-256 manifest entries independently verified, and 39 raw evidence files sealed read-only. The marker path, TimescaleDB exports, exact-window service logs, kernel logs, statistics, and active-state cleanup were also verified. Static/live recorder validation then passed.

Both smoke runs intentionally used `--skip-serial`. Counted LoRaWAN runs must pass `preflight --require-emu --require-sec` and must not skip EMU-01 source capture when the PDR denominator depends on `SENSOR_TX`

attempts.

Accepted supervised-recorder smoke - 2026-09-04

Recorder v4 was commissioned with two bounded, uncounted foreground run `--skip-serial` smokes against the live Gateway-01 plus three-node cloud. The first run, `collector-supervised-smoke-20260904-0456z`, proved collector supervision and clean automatic finalization but deliberately remains diagnostic evidence because it exposed that its first implementation timestamped supervisor health with workstation UTC while start/end boundaries used server-authoritative UTC. The workstation was materially behind the cloud clock, so those control-plane rows could appear before the run start. The same run also showed that uncounted metadata must not claim the frozen counted EMU profile when serial capture is skipped.

Both provenance defects were corrected before acceptance. `control-plane-health.csv` now records server-authoritative UTC, the bracketing workstation midpoint, measured server-minus-workstation offset, and clock-probe round-trip time. `--skip-serial` metadata now records `serial_capture_enabled=false`, `emu_profile=NOT_CAPTURED_UNCOUNTED`, and leaves counted source/HEX hashes blank rather than implying the counted firmware was observed.

The corrected run `chapter4-results/smoke/collector-supervised-clockfix-20260904` is the accepted v4 supervisor smoke. It exited 0, finalized as `RECORDED_UNCLASSIFIED`, produced 76 manifest entries with zero missing files and zero SHA-256/size mismatches, left no `_recorder-active.json`, `stop-errors.txt`, or `supervisor-errors.txt`, and recorded three monotonically increasing `PASS` supervisor rows inside the authoritative start/end window. The clock probes measured the server about 104.65-104.90 s ahead of the workstation with roughly 1.06-1.22 s round-trip time; this skew is now explicit evidence rather than a hidden timing assumption. A subsequent live preflight `--require-emu --require-sec` also passed with EMU-01 on COM11, SEC on COM16, `u1c-01` leader, `u1c-02/u1c-03` replicas, and all required evidence roles ready.

Counted-profile post-flash sanity - 2026-09-03

The uncounted readiness run `counted-profile-postflash-sanity-20260903` exercised the frozen 15-second EMU-01 image with real USB source capture and the commissioned Gateway-01/LTE/cloud path. The sealed recorder-v1 bundle contains 189 healthy source attempts (`send_status=0`, sequences 7..195), 192 unique database event keys, 2,496 normalized measurements (exactly 13 per event), and 192 Fabric-outbox rows left pending because the external Fabric worker remains intentionally disabled.

A naïve comparison of 192 DB rows against 189 captured source attempts produced an impossible 101.587% value in the sealed v1 derived summary. That number is **not a valid PDR and must not be cited**. Byte-for-byte reconstruction of each 46-byte payload proved 185 exact source/DB matches for sequences 7..191. Those matches establish a stable DB-minus-workstation timestamp offset of about +103.24 seconds. After applying that measured offset to define the source attempts actually covered by the DB export window, all 185 eligible source attempts have exact DB matches: aligned PDR = 100.0%. DB-only rows are the six current-session frames emitted before serial capture plus one prior-session sequence-8 frame admitted by the skewed wall-clock start boundary; source-only sequences 192..195 project beyond the DB export end.

Recorder version 2 therefore no longer calculates PDR as `DB-row-count/source-attempt-count`. It reconstructs the payload-v2 bytes from every `SENSOR_TX` line, correlates them exactly to `raw_data`, estimates the workstation/cloud clock offset from those exact matches, and limits the denominator to source attempts whose projected DB timestamps fall inside the actual export window. If that aligned observation window cannot be established, PDR is withheld rather than guessed. The sealed v1 bundle is intentionally left unchanged; corrected interpretation is recorded in the preflight analysis outside the sealed run.

The same sanity run exposed a Windows-specific shutdown defect: `taskkill /T /F` could hang while traversing the process tree. Direct `taskkill /PID <pid> /F` was proven against the live recorder-owned processes and a disposable child. Both Python and legacy PowerShell recorder paths now omit `/T`; the Python path also bounds termination to five seconds.

Result shape

```
chapter4-results/<group>/<run-id>/
raw/
  emu-01-source.log
  sec-source.log           # when requested
  gateway.log
  gateway-resource.csv
  ulc01-host-resource.csv
  ulc01-docker-resource.csv
  ulc02-host-resource.csv
  ulc02-docker-resource.csv
  ulc03-host-resource.csv
  ulc03-docker-resource.csv
  uplinks.csv
  measurements.csv
  fabric-outbox.csv
  *-chirpstack.log
  *-mosquitto.log
  *-node-red.log
  *-kernel.log
  markers.csv
metadata/
  run-meta.json
  start-utc.txt
  end-utc.txt
  *-snapshot-start.txt
  *-snapshot-end.txt
  database-roles-end.json
  SHA256SUMS.csv
derived/
  run-summary.json
  run-summary.csv
  trial-results.csv
  run-status.txt
```

PASS, FAIL, INVALID, and BLOCKED remain research classifications. The recorder deliberately ends a clean capture as RECORDED_UNCLASSIFIED; evidence collection must not invent the experiment outcome.

Usage

From the repository root:

```
$R = './lorawan-network-server-gateway/test/automation/research-recorder/research_recorder.py'

# Remote collectors + Patroni topology; add both flags before LoRaWAN security testing.
python $R preflight --require-emu --require-sec

# Preferred counted-run mode. Keep this terminal open; Ctrl+C finalizes safely.
# Use a second terminal for mark/trial/serial-control commands while this runs.
python $R run `
  --group authentication `
  --run-id auth-lorawan-A1-01 `
  --condition 'registered device with correct OTAA credentials' `
  --expected 'allow; JoinAccept and accepted uplink' `
  --scope lorawan `
  --capture-sec

# Timestamp a physical/software event without stopping capture.
python $R mark --message 'A1-01 reset issued'

# Read the current aggregate verifier/checkpoint state without starting a run.
# This is intentionally strict: on pre-v3 live wrappers it reports that the server-side upgrade is required.
python $R evidence-status

# Optional explicit active-recorder view; the foreground supervisor already gates health every 15 seconds.
python $R status
```

```

# When a counted OTAA trial requires reloading the exact frozen EMU image,
# keep the recorder and all remote collectors alive while intentionally yielding COM11.
# The serial log records RECORDER_SERIAL_PAUSED / RECORDER_SERIAL_RESUMED and then
# automatically reacquires the board after DFU re-enumeration.
python $R serial-pause --device emu
# reload the already-frozen counted-test ZIP; do not rebuild here
python $R serial-resume --device emu

# The same mechanism may yield SEC during A2/A3 fixture commands without stopping
# the rest of the counted evidence capture:
# python $R serial-pause --device sec
# ... issue only the documented SEC test-fixture commands ...
# python $R serial-resume --device sec

# Record the trial classification after its evidence is inspected.
python $R trial `
  --trial-id A1-01 `
  --condition 'correct OTAA credentials' `
  --expected allow `
  --actual allow `
  --status PASS `
  --notes 'JoinRequest received; JoinAccept; application uplink stored'

# Normally press Ctrl+C in the foreground `run` terminal when complete.
# It automatically exports DB/log evidence, calculates statistics, hashes files and seals raw evidence read-only.

# Recovery only: if a detached or interrupted run left _recorder-active.json,
# inspect it first, then finalize and preserve it rather than deleting it.
python $R status
python $R recover

```

For a bounded **uncounted** collector smoke test, run `--skip-serial --duration-seconds 60` is appropriate. In that mode `run-meta.json` explicitly marks serial capture disabled and the EMU profile as `NOT_CAPTURED_UNCOUNTED`; it does not claim that the frozen counted-test image was observed. Counted runs normally omit `--duration-seconds` and run until operator Ctrl+C.

Use `--interval 1` for flooding and normally `--interval 5` for baseline/resilience/resource observations. The default supervisor health interval is 15 seconds; do not make it aggressively shorter because each gate also checks the replicated evidence path.

`--skip-serial` exists only for clearly uncounted recorder smoke tests. Do not use it for a counted run whose delivery/PDR denominator depends on EMU-01 `SENSOR_TX` attempts.

Statistical interpretation

The recorder derives only metrics supported by its actual evidence. Formal Chapter 3 PDR and source-to-database delivery are deliberately separate:

```

formal Chapter 3 PDR = unique accepted EMU-01 ChirpStack deduplication IDs across both HA ChirpStack logs /
captured scheduled SENSOR_TX attempts x 100
database delivery    = exact DB payload matches for eligible source attempts / eligible source attempts x 100

```

For formal PDR, `summarize-run.py` strips ANSI log formatting, unions accepted `deduplication_ids` from `ulc01-chirpstack.log` and `ulc02-chirpstack.log`, associates them with the run's `dev_eui` from `run-meta.json`, and divides that unique accepted count by every captured scheduled `SENSOR_TX` attempt in the same run window. A scheduled source-side send failure remains in the denominator. The percentage is withheld when required logs/device identity/source attempts are missing or when accepted uplinks exceed the source-attempt boundary. This is the Chapter 3 LoRaWAN/ChirpStack metric boundary.

For database delivery, source and database evidence are correlated by reconstructing the exact 46-byte payload-v2 bytes from each `SENSOR_TX` line and matching those bytes to `telemetry.uplinks.raw_data`. Exact matches also provide the measured DB-minus-source clock offset. A source-side attempt remains in the delivery-correlation denominator only when the recorder can prove that attempt lies inside the DB observation window. Boundary attempts outside that comparable window are reported as unmatched correlation evidence, not silently counted as packet loss.

Never copy database delivery into a Chapter IV PDR column merely because both percentages may happen to be equal in a healthy run. An MQTT redelivery is not a second LoRaWAN delivery, and a second LoRaWAN session reusing a sequence/FCnt value is not collapsed merely because the counter repeats.

For replay/spoofing, only attack transmissions whose RAK5146 reception is proven belong in the counted attack denominator. `sec02_replay.py capture --test-sequence <N>` may now bind a replay/spoofing fixture to the exact accepted EMU-01 `test_sequence` designated as that trial's legitimate control; the helper still requires the configured minimum FCnt age and independently correlates the selected packet to Gateway-01 journal, packetflow, and RAK5146 reception evidence before it will create the fixture. This prevents a nearby historical uplink from being silently substituted for the intended control.

`application_event_to_db_ms` is `telemetry.uplinks.received_at - telemetry.uplinks.time`. Do not relabel it sensor-to-database latency unless a separate run proves the source/laptop UTC timestamp is correlated closely enough with the server clock.

The post-run summarizer also reports sample SD, min, max, RSSI, SNR, frequency counts, sequence/FCnt gaps, measurement counts, Fabric outbox states, and per-node host/container resource statistics.

What remains physical

Evidence capture, exports, statistics, hashing, most attack generators, and server/network fault injection can be automated. Human action remains only where the experimental condition itself is physical, such as reconnecting actually disconnected hardware, deliberately moving nodes/antennas, or changing a real environmental sensor condition.

A physical reset can be timestamped with `mark`. If a test later proves a safe software-reset mechanism for the exact board mode, a test-specific driver can automate that reset as well.

Supporting runbook: 00-README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Dissertation Testing Track

This directory is the complete operator path for preparing and executing the Chapter III/IV LoRaWAN dissertation experiments.

Current methodology authority: use [ZACARIAS-CHAPTER3-TEST-METRICS-SOURCE-OF-TRUTH.md](#) for the formal research tests, conditions, counts, durations, required metrics, evidence rules, and stated acceptance/security criteria extracted from the new chapters/Zacarias_Chapter3.pdf. Architecture/topology/deployment prose in that PDF is not current infrastructure truth; execute those tests against the verified commissioned system.

Use this track for the counted research tests only. Do not add production HA, dashboards, or unrelated services to the measured VM unless the research methodology is intentionally changed.

Directory structure

```
test/
├── 00-README.md
├── automation/
│   └── research-recorder/ # supervised evidence capture, preflight, sealing, recovery, summaries
├── preparation/
│   ├── 00-README.md
│   ├── gateway/      # Raspberry Pi 4B + RAK5146
│   ├── server/       # Ubuntu VM + minimum seven-service stack
│   ├── sensor/       # RAK4631 EMU-01 + SEC-02
│   │   ├── assembly/
│   │   └── preflight/ # uncounted final sensor -> ChirpStack -> data-path GO/NO-GO
│   └── tools/        # test laptop + generators + resource logging
└── execution/
    ├── 00-README.md
    ├── 01-common-run-preparation.md
    ├── 02-normal-operation.md
    ├── 03-authentication-access-control.md
    ├── 04-replay-spoofing.md
    ├── 05-data-integrity.md
    ├── 06-traceability.md
    ├── 07-dos-flooding.md
    ├── 08-resilience-recovery.md
    └── 09-results-and-completion.md
```

What is actually tested

The sensor **measurement values are real physical Agriculture Kit readings**. The packet-accounting sequence is controlled, while the LoRaWAN path remains physical and real:

```
RAK4631 EMU-01 full physical Agriculture Kit sensor node
-> real AS923 LoRaWAN RF
-> Raspberry Pi 4B + RAK5146
```

```
-> Concentrator  
-> MQTT Forwarder  
-> gateway local Mosquitto buffer  
-> mTLS  
-> server Mosquitto  
-> ChirpStack  
-> Node-RED  
-> PostgreSQL / TimescaleDB  
-> Fabric outbox  
-> Fabric adapter  
-> OpenBao  
-> external Hyperledger Fabric
```

EMU-01 is the legitimate full-sensor OTAA device. It carries one complete Agriculture Kit sensor set and adds deterministic packet-sequence markers to real measurements. SEC-02 first verifies every second-copy sensor, then becomes the isolated invalid-device/raw-RF security node and must never contain EMU-01's legitimate AppKey or legitimate session keys.

One-page operator journey

Follow this sequence. Do not jump directly to an experiment.

1. PREPARE GATEWAY 01-03
Assemble -> Gateway OS -> AS923 Concentrator -> record real Gateway EUI
2. PREPARE SERVER
Create VM -> deploy minimum seven services -> register that EUI -> issue mTLS identity/ACL
3. FINISH GATEWAY 04-06
Local buffer -> MQTT Forwarder -> mTLS -> ChirpStack Last seen -> real RF acceptance
4. PREPARE SENSOR NODES
Assemble two RAK4631 nodes -> install/verify every Agriculture Kit sensor -> keep the complete A-set on EMU-01 -> verify the B-set on SEC-02 -> configure physical-sensor OTAA telemetry -> configure SEC-02 invalid OTAA/raw RF -> prove RAK5146 receives SEC-02
5. PREPARE TEST TOOLS
Test laptop -> source capture -> flood generators -> server/gateway resource logging
6. SENSOR PREFLIGHT - UNCOUNTED
Final hardware/firmware -> 10 healthy cycles -> RAK5146 -> OTAA/ChirpStack -> decoder -> Node-RED/TimescaleDB/Fabric -> GO/NO-GO
7. FREEZE THE TESTBED / TRANSITION
Require SENSOR_PREFLIGHT_STATUS=GO. Record versions/configuration. No unplanned changes inside a repetition group.
8. EXECUTION 01
Start fresh counted-run IDs, clocks, source/resource captures, and one known-good control.
9. RUN BASELINE
Three 30-minute normal-operation runs
10. RUN SECURITY / INTEGRITY / TRACEABILITY / FLOODING / RESILIENCE TESTS
One controlled condition at a time
11. CALCULATE RESULTS
Raw evidence -> validated trial set -> Chapter IV summaries

Required preparation order

The gateway and server have one deliberate dependency: the server needs the **real Gateway EUI** before it can issue the gateway mTLS identity.

1. Open [preparation/00-README.md](#).
2. Complete Gateway 01-03 and record the real Gateway EUI.
3. Complete server preparation and provision that exact EUI in ChirpStack, the broker certificate, and ACL.
4. Return to Gateway 04-06 and complete end-to-end gateway acceptance.
5. Assemble both RAK4631 boards using [preparation/sensor/assembly/00-README.md](#), then configure EMU-01 and SEC-02.
6. Prepare the separate test laptop and measurement tools.
7. Run the dedicated [sensor preflight](#) against the frozen gateway/server path.
8. Require SENSOR_PREFLIGHT_STATUS=G0 and complete the final preparation gate.
9. Enter [execution/01-common-run-preparation.md](#) for fresh counted-run capture.

For full-stack counted testing, do not start until every preparation item passes. The intentionally narrower LoRaWAN security exception is defined only in [execution/00-README.md](#) and may proceed only when LORAWAN_TRACK_STATUS=G0; do not widen that exception to Fabric-dependent experiments.

Use [automation/research-recorder/README.md](#) for the accepted automated evidence-capture workflow. Preserve recorder outputs under chapter4-results/; smoke, interrupted, failed, and invalid runs are evidence and are not cache directories.

Minimum test VM

Physical host: 8 GiB RAM / 8 CPU threads
 Test VM: 5 GiB RAM / 4 vCPU / 50 GiB SSD-backed disk
 OS: Ubuntu Server 24.04 LTS, no desktop GUI

The minimum server stack is exactly:

Mosquitto
 Valkey
 ChirpStack
 PostgreSQL / TimescaleDB
 Node-RED
 OpenBao
 Fabric adapter

Authoritative experiment inventory

The new Zacarias Chapter 3 expands the formal test set. The concise inventory below is mandatory; use the source-of-truth document for full metric definitions and evidence rules.

P1	End-to-end normal operation:	3 x 30-minute runs
P2	Fabric performance/overhead:	4 workloads x 3 repetitions = 12 x 5-minute runs
R1	External-Internet interruption/recovery:	3 x 2-hour runs (30 normal + 60 outage + 30 recovery)
R2	Fabric consistency/reconciliation:	10 control + 10 Fabric-unavailable; reconcile the same 10 pending
A1	Authentication/access control:	90 attempts
A2	Fabric endorsement-policy enforcement:	30 attempts
S1	LoRaWAN replay/spoofing:	40 counted attempts
S2	Application-layer duplicate/replay:	listed by Chapter 3 Table 3.4, but count/procedure/metrics are unspecified -> methodology resolution required
F1/F2	DoS/flooding:	18 x 5-minute load runs + 5-minute recovery after each
I1/I2	Application/post-storage integrity:	40 attempts
I3	Blockchain duplicate/overwrite:	10 baseline anchors + 10 duplicate + 10 overwrite attempts
T1/T2	Traceability:	20 trials / 60 records

Do not collapse R1 and R2: Internet interruption and Fabric unavailability/reconciliation are separate experiments. Likewise, P2, A2, and I3 are required formal tests and must not be omitted simply because older execution indexes did not list them. S2 is a chapter-internal methodology gap: it is explicitly listed but not defined in the later procedure, so it must remain visible and unresolved rather than being silently dropped or assigned invented counts.

The rule for every counted trial

Every trial must answer all five questions:

1. What exact condition did we intentionally create?
2. How do we prove the condition actually reached the layer being tested?
3. What was the expected decision before the trial started?
4. What did the system actually do?
5. Which raw file/log/record proves the result?

If question 2 cannot be proven, the trial is **INVALID**, not PASS. If evidence for question 5 is missing, the trial is **INVALID** and must be rerun.

Evidence rule

Preserve the EMU-01 source log, timestamps, stable identifiers, gateway/ChirpStack decision evidence, database exports, Fabric evidence, and required CPU/memory samples. Screenshots may support a result but must not replace raw logs or CSV/database evidence.

Canonical source: [deployment/technology-guides/21-end-to-end-integration-and-data-lineage.md](#). For any mutable production-status statement, check the dated baseline and verified live state.